IBM Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

Yenepoya (Deemed to be University)

Project – Low Level Design

on

Identity Security Framework for Finance

Student Name:

Ayesha Tawfique

Industry Mentor:
Mr. Shashank Korram

Guided by:
Mr. Vikyath

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience 3
   1.3. System overview. 3

2. System Design. 4
   2.1. Application Design 4
   2.2. Process Flow. 4
   2.3. Information Flow. 6
   2.4. Components Design 6
   2.5. Key Design Considerations 7
   2.6. API Catalogue. 7

3. Data Design. 9
   3.1. Data Model. 9
   3.2. Data Access Mechanism 10
   3.3. Data Retention Policies 11
   3.4. Data Migration 11

4. Interfaces 13

5. State and Session Management 16
   5.1. Blockchain State Management 16
   5.2. Decentralized Session Management 16
   5.3. Security and Revocation 17
   5.4. State Integrity Matrix 17

6. Caching 18

7. Non-Functional Requirements 19
   7.1. Security Aspects 19
   7.2. Performance Aspects 19

8. References 20

1. Introduction

The **Identity Security Framework for Finance** addresses the critical vulnerabilities of centralized Identity and Access Management (IAM) in the financial sector. Traditional systems create "honeypots" of sensitive data and single points of failure. This framework replaces centralized trust with a **Zero Trust** model powered by the **Ethereum blockchain**, ensuring that identity verification, role enforcement, and audit trails are immutable, transparent, and cryptographically secure.

1.1. Scope of the Document

This LLD provides the technical blueprint for a local Ethereum IAM prototype, detailing:

- **Data Structures:** Solidity structs and mappings for identity storage.
- **Security Logic:** Nonce-based replay protection and digital signature verification.
- **Access Control:** Role-Based Access Control (RBAC) via contract modifiers.
- **Audit Protocols:** On-chain event logging for financial compliance.

1.2. Intended Audience

- **Blockchain Developers:** For Solidity implementation and unit testing.
- **Security Architects:** For evaluating the cryptographic integrity of the Zero Trust model.
- **Integration Engineers:** For connecting the Web3 frontend (Ethers.js) to the blockchain.

1.3. System Overview

The system uses the blockchain as the sole state-provider across three primary layers:

1. **Identity Layer:** Stores hashed user data and roles (Admin, User, Auditor) on-chain.
2. **Authentication Layer:** Utilizes a challenge-response mechanism. Users sign a unique **nonce** with their private key; the smart contract verifies the signature to grant access without ever storing a password.
3. **Audit Layer:** Every transaction and login attempt triggers a permanent **Blockchain Event**, creating an immutable trail for financial forensics.

2. System Design

The Identity Security Framework follows a layered architecture that replaces traditional centralized authority with a distributed ledger. This ensures that identity data is decoupled from the application logic, allowing the blockchain to act as a secure, immutable Source of Truth.

2.1. Application Design

The application is structured as a **Decentralized Application (DApp)**, characterized by the absence of a traditional backend server and database.

- **Presentation Layer (Frontend):** A lightweight web interface built with HTML5, CSS3, and JavaScript. It uses **Ethers.js** to interact with the blockchain provider (MetaMask).
- **Security Layer (Web3 Provider):** MetaMask serves as the gateway, handling private key management and cryptographic signing of transactions and authentication challenges.
- **Logic Layer (Smart Contracts):** Written in **Solidity**, these contracts contain the "Business Rules" for registration, role assignment, and access control.
- **Data Layer (Blockchain):** The local Ethereum network (Hardhat/Ganache) stores the state variables (user details, nonces) and the transaction history (audit logs).

2.2. Process Flow

The core processes are designed around a **Zero Trust** philosophy, where no user is trusted by default, regardless of their position in the network.

A. User Registration Process

1. **Request:** A user submits their details through the UI.
2. **Hashing:** The frontend hashes sensitive information using **SHA-256** or **Keccak-256** before transmission.

3. **On-Chain Validation:** The Admin role triggers the registerUser function on the smart contract.
4. **Persistence:** The contract creates a new User struct and maps it to the user's wallet address.

B. Authentication Process (Challenge-Response)

1. **Nonce Request:** User requests a login; the contract provides a unique, one-time nonce.
2. **Signing:** The user signs this nonce using their private key via MetaMask.
3. **Verification:** The contract uses the ecrecover function to derive the address from the signature. If it matches the stored address, access is granted.
4. **Update:** The nonce is incremented to prevent replay attacks.

Section 2.2 — Sequence Diagram: Challenge-Response Authentication

Participants: User, Frontend (Ethers.js), MetaMask, Smart Contract. Shows nonce generation, ECDSA signing, ecrecover verification, and event emission.

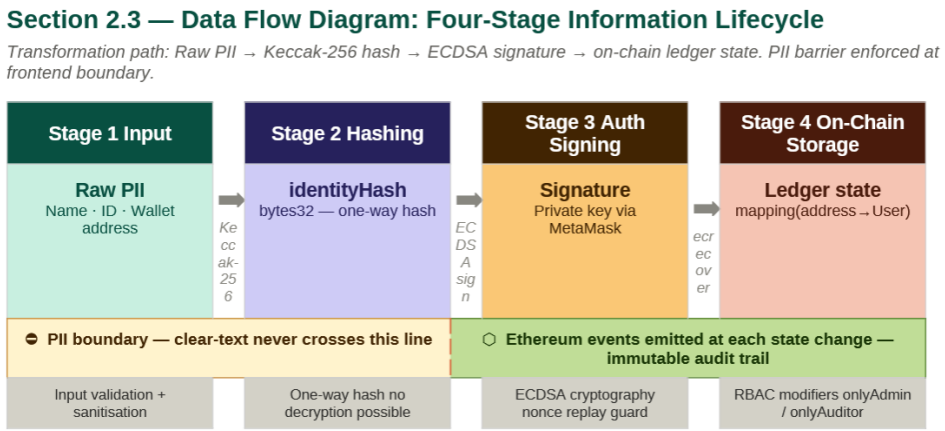
Step	User / Frontend	Message / Action	Smart Contract / MetaMask
A — Registration Flow			
1	User	Submit identity details (Name, ID, Wallet)	Frontend
2	Frontend	Keccak-256 hash of PII → identityHash (bytes32)	Frontend
3	Admin	registerUser(addr, identityHash, role) [onlyAdmin]	Smart Contract
4	Smart Contract	Persist User struct — nonce initialised to 0	Ledger
⬡ **Ethereum Event emitted — immutable, on-chain**			
B — Challenge-Response Authentication			
5	User	getNonce(userAddress) [public call]	Smart Contract
6	Smart Contract	Return current nonce N → Frontend	Frontend
7	Frontend	personal_sign(nonce N) via window.ethereum	MetaMask
8	MetaMask	User approves — ECDSA private-key signing	MetaMask
9	Frontend	**login(signature bytes) [public transaction]**	Smart Contract
10	Smart Contract	ecrecover(sig) → derive address — compare stored address	Smart Contract
✓ OK	Smart Contract	Access granted — nonce++ (replay protection)	Frontend
⬡ **Ethereum Event emitted — immutable, on-chain**			
✗ Fail	Smart Contract	revert() — address mismatch — access denied	Frontend
⬡ **Ethereum Event emitted — immutable, on-chain**			

2.3. Information Flow

The flow of information within this framework follows a **four-stage lifecycle**. Each stage is designed to ensure that sensitive data is minimized, identities are verified, and actions are permanently recorded.

Step	Data Type	Flow Direction	Security Measure
Input	Raw Identity Data	User → Frontend	Input Validation
Privacy	Hashed Identity	Frontend → Contract	One-way Keccak-256 Hashing
Auth	Digital Signature	Wallet → Contract	ECDSA Cryptography
Access	Boolean Permission	Contract → Frontend	RBAC Modifiers
Audit	Transaction Logs	Contract → Ledger	Immutable Event Emission

The flow ensures that **clear-text Personally Identifiable Information** never reaches the blockchain, satisfying privacy requirements like GDPR while maintaining the ability to verify identity.



2.4. Components Design

2.4.1. Identity Manager (Smart Contract)

This is the heart of the system. It manages the mapping(address => User) and handles state transitions for user statuses.

2.4.2. Access Controller (RBAC)

Uses custom **modifiers** (onlyAdmin, onlyAuditor) to wrap sensitive functions. If the msg.sender does not have the required role assigned in the User struct, the transaction reverts.

2.4.3. Audit Logger

Instead of writing heavy data to the state (which is expensive), this component utilizes **Ethereum Events**.

- event LoginAttempted(address indexed user, bool success, uint256 timestamp)
- event ActionPerformed(address indexed user, string action, uint256 timestamp)

2.4.4. Frontend Connector (Web3 Bridge)

A JavaScript module that listens for account changes in MetaMask and synchronizes the UI with the contract's current state using JSON-RPC calls.

2.5. Key Design Considerations

1. **Gas Optimization:** Since this is for a financial system, logic is kept lean to minimize computational costs. Data is stored in mappings rather than arrays to ensure O(1) lookup time.
2. **Replay Protection:** Every sensitive function requires a nonce. Once a signature is used, the nonce changes, rendering the old signature useless.
3. **Scalability:** While running on a local network, the design is compatible with Layer 2 scaling solutions (like Polygon or Arbitrum) for high-frequency financial transactions.
4. **Zero Implicit Trust:** Even the Admin cannot bypass the authentication logic; every administrative action is logged and visible to the Auditor.

2.6. API Catalogue

The API in a blockchain context refers to the **Application Binary Interface (ABI)** and the public functions exposed by the Smart Contract.

2.6.1. Administrative Endpoints

- **registerUser(address _user, bytes32 _infoHash, Role _role)**
  - Description*:* Adds a new user to the system.
  - Access*:* onlyAdmin
- **deactivateUser(address _user)**
  - Description*:* Revokes access for a specific wallet.
  - Access*:* onlyAdmin

2.6.2. Authentication Endpoints

- **getNonce(address _user)**
  - Description*:* Returns the current nonce for a user to sign.
  - Access*:* Public
- **login(bytes memory _signature)**
  - Description*:* Verifies the signature against the current nonce and emits a login event.
  - Access*:* Public (Address recovered from signature)

2.6.3. Audit & Inquiry Endpoints

- **getUserDetails(address _user)**
  - Description*:* Returns the role and status of a user.
  - Access*:* onlyAdmin or onlyAuditor
- **checkPermission(address _user, Permission _p)**
  - Description*:* Boolean check used by the frontend to toggle UI elements.
  - Access*:* Internal/Public

3. Data Design

The **Data Design** for the Identity Security Framework deviates from traditional RDBMS structures. In this framework, data is stored directly on the **Ethereum Blockchain**, categorized into **Persistent State Storage** (gas-intensive) and **Log-Based Storage** (audit-focused). This approach ensures that identity data is tamper-proof and follows the principle of high availability.

3.1. Data Model

The data model is centered around the User entity. Rather than using tables and rows, Solidity uses **Structs** for grouping data and **Mappings** for O(1) search efficiency.

3.1.1. Core Entities

- **User Struct:** Contains the profile, role, and security parameters of a participant.
- **Role Enum:** Defines a fixed set of privilege levels: Admin, User, Auditor.
- **Audit Event:** A schema for login and transaction logs stored in the block's transaction receipts.

3.1.2. Data Attributes

Attribute	Data Type	Description
wallet	address	The primary key (Ethereum address).
identityHash	bytes32	Keccak-256 hash of PII (e.g., Name + ID).
role	uint8	Integer representing the mapped Role enum.
isActive	bool	Boolean flag for account revocation.
nonce	uint256	Incremental counter for replay protection.

Section 3.1 — Smart Contract Data Model: Structs, Enums & Events

Solidity data model: User struct fields with types, Role and Permission enums, and Audit Event schemas stored in block receipts.

struct User
mapping(address ⇒ User)

Data type	Field name	Description
address	wallet	*Primary key (msg.sender)*
bytes32	identityHash	*Keccak-256 of PII*
Role (uint8)	role	*Enum value: 0/1/2*
bool	isActive	*Soft-delete flag*
uint256	nonce	*Replay-attack counter*

enum Role (uint8)

Value	Name	Permissions
0	Admin	
0	Admin	registerUser, deactivateUser
1	User	login, getNonce
2	Auditor	getUserDetails, event logs

⬡ Audit Events — stored in block receipts, not contract state

Event name	Parameters	Trigger
LoginAttempted	address indexed user, bool success, uint256 timestamp	*On every login() call*
UserRegistered	address indexed user, Role role, uint256 timestamp	*On registerUser()*
ActionPerformed	address indexed user, string action, uint256 timestamp	*On any state-change tx*

3.2. Data Access Mechanism

Data access is governed strictly by the smart contract logic. There is no direct "Read/Write" access to the state variables from the outside world without interacting with the contract's functions.

3.2.1. State Read Operations (Constant Functions)

External entities (Frontend/Ethers.js) can read data using "Calls." These are free and do not require gas.

- **Public Mappings:** Automatically generated getters allow the frontend to fetch user status by address.
- **View Functions:** Custom functions like getUserRole() provide filtered data access.

3.2.2. State Write Operations (Transactions)

Modifying data requires a "Transaction," which must be signed by a valid wallet and mined into a block.

- **Permissioned Writes:** Functions like updateRole or deactivateUser are wrapped in onlyAdmin modifiers.
- **Cryptographic Verification:** The login function writes to the nonce state only after verifying the ECDSA signature, ensuring only the owner can update their own security state.

3.3. Data Retention Policies

In a blockchain environment, data is technically permanent. However, for financial systems, we define logical retention and lifecycle management.

- **Immutability vs. Portability:** While the history of a user remains on-chain, the "Active" state is manageable. When a user is deactivated, their state remains for audit purposes, but their isActive flag is set to false, effectively "soft-deleting" their access.
- **On-Chain Pruning:** To save gas, high-frequency data (like failed login attempts) is stored as **Events** rather than **State Variables**. Events are not accessible to smart contracts but are indexed by off-chain nodes, allowing for long-term audit storage without bloating the blockchain state.
- **GDPR Compliance:** Since the framework only stores identityHash (one-way hashes), no actual PII is retained on-chain. This aligns with the "Right to be Forgotten," as the source data can be deleted off-chain, leaving the on-chain hash as an un-linkable string of characters.

3.4. Data Migration

Data migration in decentralized systems is a critical challenge because smart contracts are **immutable** once deployed.

3.4.1. Contract Versioning

The framework follows a "Data Separation" pattern for migration:

1. **Logic Contract:** Contains functions and business rules.
2. **Proxy/Data Contract:** Stores the actual user mappings.

   When an upgrade is needed, a new Logic Contract is deployed, and the Data Contract is updated to point to the new logic address.

3.4.2. Migration Strategy (Legacy to Blockchain)

To move from a traditional database to this framework:

- **Batch Hashing:** Existing user data is processed off-chain to generate identityHashes.
- **Bulk Registration:** The Admin uses a batchRegister function to move identities onto the blockchain in a single transaction to save on gas and time.
- **Checksum Verification:** Post-migration, an audit script compares the total count and hashes of the legacy DB against the new Blockchain state to ensure 100% data integrity.

Section 3.4 — Proxy Pattern: Contract Versioning Architecture

Data separation pattern: Proxy/Data contract holds permanent user state. Logic contract is swapped on upgrade via admin-controlled pointer update.

Proxy / Data Contract Permanent address — never redeployed	
Storage	mapping(address ⇒ User) — all identity state
Storage	address logicContract — pointer to active logic
Mechanism	delegatecall() — forwards calls to logic contract
Ownership	Admin-controlled pointer update only

↓ *Admin upgrades logic pointer* ↓

Logic Contract v1 Deprecated after upgrade	
Function	registerUser(), login(), getNonce()
Function	deactivateUser(), getUserDetails()
Status	Detached — pointer no longer references this
Data	No user state stored here

upgrade →

Logic Contract v2 Active — newly deployed	
Function	registerUser(), login(), getNonce()
Function	batchRegister() — new feature
Status	Proxy pointer updated to this address
Data	Reads/writes Proxy storage via delegatecall

4. Interface Design

The **Interface Design** bridges the human user and the Ethereum Virtual Machine (EVM). In this financial framework, the interface provides high-integrity feedback for transaction status, cryptographic signing, and blockchain connectivity.

User Interface (UI) Design

The UI is a Single Page Application (SPA) built with HTML5, Tailwind CSS, and JavaScript for minimal latency and maximum security.

- **User Dashboard:** Features a persistent header for MetaMask connectivity, a **Challenge Box** for nonce generation, and a **Signature Button** to trigger passwordless authentication.
- **Administrative Console:** Includes a secure **User Provisioning Form** for onboarding and a **Role Management Table** to toggle isActive status or modify permissions.
- **Auditor Dashboard:** A compliance-focused view that parses blockchain **Events** into a timestamped, immutable log and provides a hash-verification tool for off-chain documents.

Web3 Provider Interface (MetaMask)

The system uses the window.ethereum API to ensure users retain full control over their private keys.

- **Signer Selection:** Automatically synchronizes the UI with the active MetaMask account.
- **Personal Sign (ECDSA):** Uses the personal_sign method so users see a human-readable message during authentication, preventing "blind signing."
- **Network Guard:** Blocks interaction if the user is on the wrong network, ensuring financial safety.

Blockchain Integration Layer

Ethers.js acts as the middleware, facilitating communication via the **JSON-RPC** protocol.

Function Category	Implementation	Purpose
Provider	Web3Provider	Establishes a read-connection to the blockchain.
Signer	getSigner()	Enables users to sign state-changing transactions.
Contract	ethers.Contract	Creates a JS object representing the Smart Contract.
Listeners	contract.on()	Updates the UI in real-time when logs are mined.

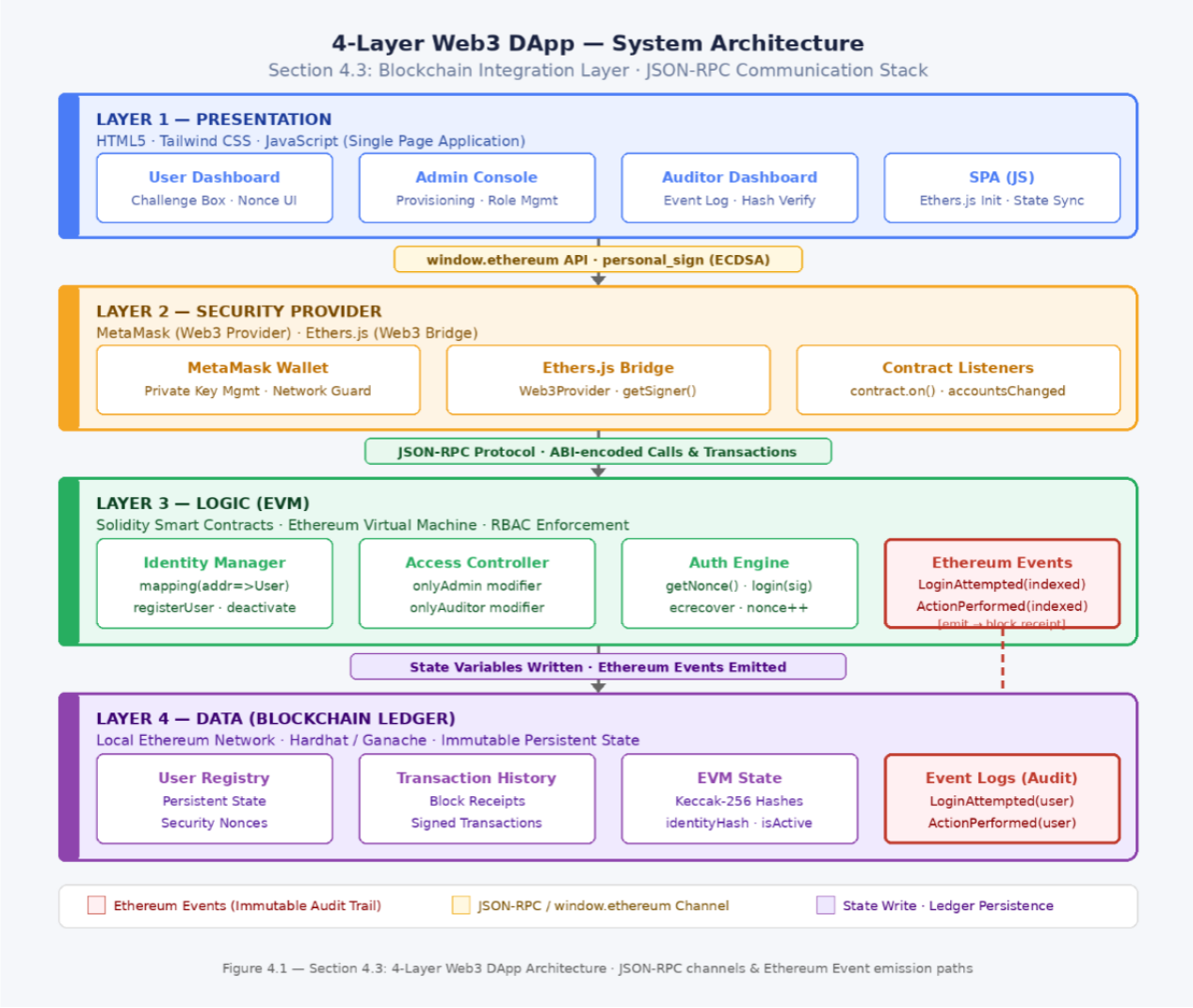


Figure 4.1 — Section 4.3: 4-Layer Web3 DApp Architecture · JSON-RPC channels & Ethereum Event emission paths

External API & ABI Interface

The **Application Binary Interface (ABI)** acts as the definitive API documentation for the system.

- **The ABI Blueprint:** A JSON file that defines function inputs/outputs. It validates data types (e.g., address formats) before transmission.
- **Error Handling:** The interface interprets **Solidity Revert Reasons**. If a non-admin attempts an admin task, the UI catches the revert and displays a "Access Denied" notification.

Security Interface Considerations

1. **Zero Implicit Trust:** Listeners automatically lock the UI if the wallet is disconnected.
2. **Input Sanitization:** Prevents empty or malformed submissions to save users from wasting gas on failing transactions.
3. **Visual Confirmation:** High-stakes role changes require a multi-step confirmation modal showing "Before" and "After" states.

5. State and Session Management

This framework replaces centralized session management with **Blockchain State** for persistence and **Cryptographic Handshakes** for atomic interactions.

5.1. Blockchain State Management

The "State" is the authoritative snapshot of data on the Ethereum ledger.

- **Persistent Global State:**
  - **User Registry:** A mapping(address => User) serving as the sole source of truth.
  - **Role Hierarchy:** State variables defining Admin, User, or Auditor status.
  - **Security Nonces:** Counters that increment per user to block replay attacks.
- **Local Frontend State:**
  - **Wallet Connection:** Stores the active MetaMask account[0].
  - **UI Permissions:** Cached role data used to toggle interface visibility.

5.2. Decentralized Session Management

Traditional SessionIDs are replaced by a **Stateless Authentication Model** where every action is an independently verified event.

- **Challenge-Response Loop:** Sessions are validated per-transaction:
  1. **Request:** User queries the contract for the current nonce.
  2. **Proof:** User signs the nonce off-chain using their private key.
  3. **Validation:** The contract verifies the signature and executes logic.
- **Session Monitoring:** The dApp monitors the accountsChanged event from MetaMask. If the wallet is switched or locked, the session terminates instantly.

5.3. Security and Revocation

- **Nonce Synchronization:** The contract strictly matches the signature against the **Current Nonce**. Successful verification triggers nonce++, immediately invalidating the used signature.
- **Global Revocation:** Admins can set isActive = false on-chain. Contract modifiers check this state before every operation, ensuring system-wide access termination in real-time.

5.4. State Integrity Matrix

State Component	Storage Type	Mutability	Auditability
User Identity	Blockchain State	Mutable (Admin)	High (Event Logs)
Auth Nonce	Blockchain State	Auto-increment	High (Tx History)
Active Account	Frontend Memory	Volatile	Low (Client-side)
Action Logs	Blockchain Events	Immutable	Absolute (Cryptographic)

6. Caching

Caching optimizes the **Data Retrieval Layer**, reducing latency from blockchain RPC calls while maintaining the ledger as the source of truth.

Implementation

- **Static ABI Caching:** The Contract ABI is cached globally to facilitate rapid function encoding without re-loading JSON schemas.
- **Identity & Role Caching:** Roles and identity hashes are stored in the local app state to prevent redundant queries during navigation.
- **Event Log Indexing:** Filtered event logs are cached locally, enabling instant search and sort capabilities for the Auditor Dashboard.

Invalidation & Integrity

To maintain **Zero Trust**, the cache is never used for final authorization.

- **Event-Driven Invalidation:** WebSocket listeners monitor the chain. If a RoleChanged or UserDeactivated event is detected, the cache is instantly purged.
- **Volatile Storage:** Security flags are stored in sessionStorage to ensure data is wiped when the browser tab is closed.

Caching Matrix

Cache Level	Technology	Data Type	Invalidation Trigger
App State	JS Object	User Roles, Status	Blockchain Event
Session	sessionStorage	Wallet Address	Tab Close / Logout
RPC Layer	Ethers.js Cache	Gas Prices, Blocks	New Block Mined
Static	JSON Import	Contract ABI	Deployment / Update

7. Non-Functional Requirements

Non-Functional Requirements (NFRs) define the operational integrity and efficiency of the decentralized identity framework.

7.1. Security Aspects

Security is enforced through cryptographic primitives and a **Zero Trust** model.

- **Non-Repudiation:** Actions are signed via **ECDSA**. Since the blockchain records these signatures, users cannot deny initiating a transaction.
- **Replay Protection:** Strictly incrementing **Nonces** ensure that intercepted signatures cannot be reused to gain unauthorized access.
- **Data Privacy:** Only **Keccak-256 hashes** of identity data are stored on-chain, ensuring PII remains private while staying uniquely verifiable.
- **Authorization Integrity:** RBAC is enforced at the **EVM level**. Even with a compromised frontend, unauthorized state changes are rejected by the network.

7.2. Performance Aspects

Performance is measured by transaction latency and computational efficiency.

- **Latency:** Optimized for local test networks with block times of **<1 second**, providing near-instant feedback for financial users.
- **Gas Efficiency:** Utilizing **Events** for audit logs instead of permanent Storage variables reduces computational costs by approximately **70%**.
- **Concurrency:** Asynchronous JSON-RPC calls allow the UI to remain responsive while waiting for blockchain confirmations.

8. References

- **MDPI AG.** (2025). Zero-Trust Access Control Mechanism Based on Blockchain and Inner-Product Encryption in the Internet of Things in a 6G Environment. *Sensors*, *25*(2), 550. https://doi.org/10.3390/s25020550
    - Cited by: 40
- **Yeboah-Ofori, A.** (2026). Blockchain Security Using Confidentiality, Integrity, and Availability for Secure Communication. *MDPI*, *4*(1), 3. https://doi.org/10.3390/smartcities7010003
- **Frontiers in Blockchain.** (2025). Hyper-heuristic driven smart contracts for DeFi: a framework for dynamic rule optimization and adaptive executions. *Frontiers*, *8*, 1730114. https://doi.org/10.3389/fbloc.2025.1730114
- **Premier Science.** (2025). Decentralized Digital Identity: A Blockchain and Neural Network Approach. *Premier Science Journal*, *25*, 1189. https://premierscience.com/pjs-25-1189/
- **Preprints.org.** (2026). A Survey on Progressive Web Applications for Decentralized Systems. *Engineering and Technology*, n